



Universidad Tecnológica Nacional
Facultad Regional General Pacheco
Base de Datos II



UTN|FRGP

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL GENERAL PACHECO

Equipo 38

Integrantes:

Pacheco, Ián Gabriel Legajo: 27687

Campos, Guillermo Legajo: 22565

Explicación del sistema:

El sistema consiste en una base de datos desarrollada en SQL Server, destinada a gestionar de forma estructurada y eficiente la información correspondiente a una biblioteca. La base de datos almacenará información sobre libros, socios, empleados, préstamos y multas. Se podrá registrar socios, consultar la disponibilidad de libros, realizar préstamos y devoluciones, registrar multas y visualizar la cantidad de multas por socio. El sistema permitirá a los empleados gestionar el catálogo, controlar los préstamos y multas, además de acceder a informes de actividad de la biblioteca.

Funcionalidades Principales:

Los socios podrán:

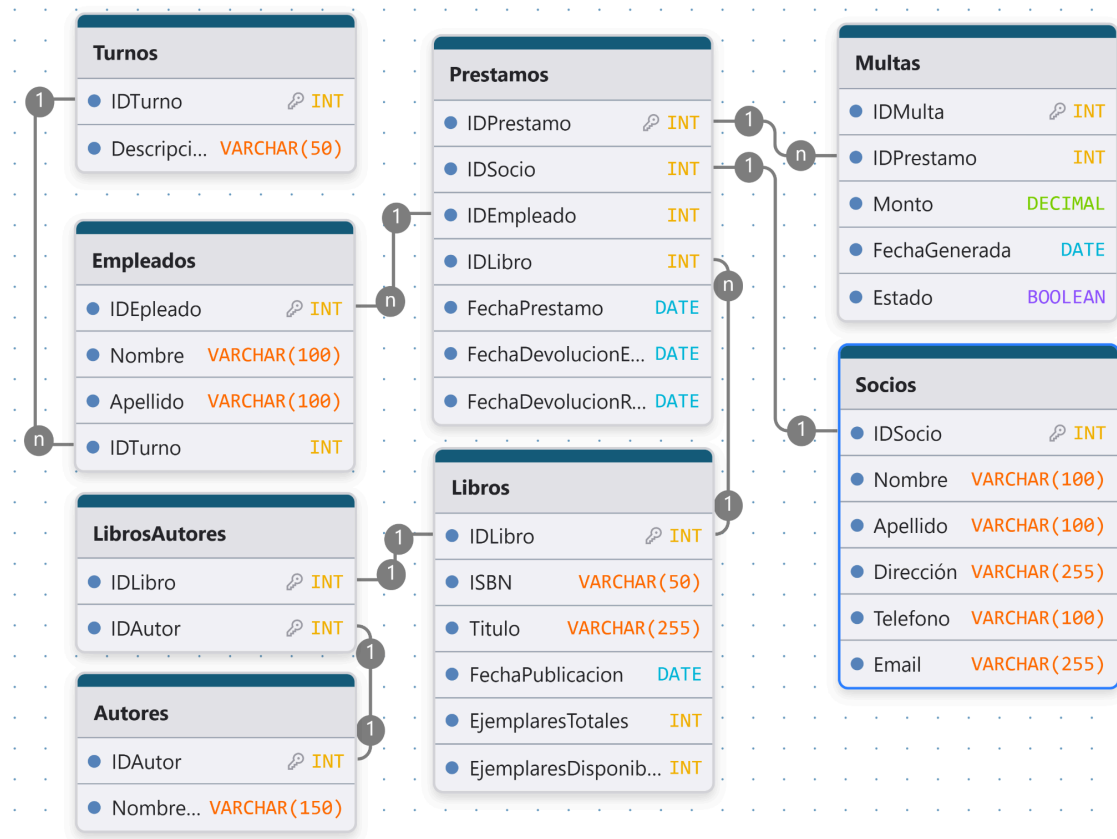
- Consultar historial de préstamos.
- Visualizar multas pendientes.

El personal de la biblioteca podrá:

- Registrar libros con su información correspondiente.
- Gestionar préstamos, devoluciones y multas por devoluciones tardías.
- Controlar el stock de ejemplares disponibles.
- Generar reportes.
- Supervisar la actividad general de la biblioteca y el estado de los préstamos.

El sistema está construido con una base de datos relacional que almacenará información sobre socios, libros, empleados, préstamos y multas. Todas estas, con sus respectivas relaciones, permitirán una gestión óptima de la biblioteca.

Diagrama Entidad-Relación (DER):



Objetos de Base de Datos clave en el Sistema:

Vista Nº 1:

Se utiliza la vista "Vista_Libros_Catalogo" para mostrar información detallada de los libros del catálogo de la biblioteca. Incluye el ISBN, título, fecha de publicación, cantidad total de ejemplares, ejemplares disponibles y sus respectivos autores.

Base de Datos 2

```
CREATE VIEW Vista_Libros_Catalogo
AS SELECT
    l.IDLibro,
    l.ISBN,
    l.Titulo,
    STRING_AGG(a.NombreAutor, ', ') AS [Autores],
    l.FechaPublicacion,
    l.EjemplaresTotales,
    l.EjemplaresDisponibles
FROM Libros l
INNER JOIN LibrosAutores la
    ON l.IDLibro = la.IDLibro
INNER JOIN Autores a
    ON la.IDAutor = a.IDAutor
GROUP BY l.IDLibro, l.ISBN, l.Titulo, l.FechaPublicacion, l.EjemplaresTotales, l.EjemplaresDisponibles;

GO
```

Vista Nº 2:

Se utiliza la vista “Vista_Prestamos_Vigentes” para mostrar un listado de los préstamos activos, e incluye el ID del préstamo, el nombre del socio, título del libro, la fecha del préstamo, y la fecha de devolución esperada.

```
CREATE VIEW Vista_Prestamos_Vigentes
AS SELECT
    p.IDPrestamo,
    s.Nombre,
    l.Titulo,
    p.FechaPrestamo,
    p.FechaDevolucionEsperada
FROM Prestamos p
INNER JOIN Socios s
    ON p.IDSocio = s.IDSocio
INNER JOIN Libros l
    ON p.IDLibro = l.IDLibro
WHERE p.FechaDevolucionReal IS NULL;

GO
```

Vista Nº 3:

Se utiliza la vista “Vista_Multas” para mostrar un listado con las multas registradas por cada socio, incluyendo la cantidad de multas y el monto total. Además incluye el ID del socio, el nombre y el apellido.

```
CREATE VIEW Vista_Multas
AS SELECT
    s.IDSocio,
    s.Nombre,
    s.Apellido,
    COUNT(m.IDMulta) AS [Cantidad de multas],
    SUM(m.Monto) AS [Monto total]
FROM Multas m
INNER JOIN Prestamos p
    ON m.IDPrestamo = p.IDPrestamo
INNER JOIN Socios s
    ON p.IDSocio = s.IDSocio
GROUP BY s.IDSocio, s.Nombre, s.Apellido;

GO
```

Vista N° 4 (Apoyada en Funciones de Usuario):

Se utiliza la vista “Vista_Atrasos” para mostrar un listado de atrasos, apoyándose en funciones de usuario para su ejecución.

```
CREATE VIEW Vista_Atrasos
AS SELECT
    p.IDPrestamo,
    s.IDSocio,
    s.Nombre + ' ' + s.Apellido AS [Nombre del Socio],
    l.IDLibro,
    l.Titulo AS [Título del Libro],
    p.FechaPrestamo,
    p.FechaDevolucionEsperada,
    p.FechaDevolucionReal,
    dbo.fn_Dias_Atraso(p.IDPrestamo) AS [Días de Atraso]
FROM Prestamos p
INNER JOIN Socios s
    ON p.IDSocio = s.IDSocio
INNER JOIN Libros l
    ON p.IDLibro = l.IDLibro;

GO
```

Procedimiento Almacenado N°1:

Se utiliza el procedimiento almacenado “sp_Agregar_Socio” para registrar a un nuevo socio en la base de datos, teniendo en cuenta que deben llenarse todos los campos, y verificando que no haya emails duplicados y que el formato sea el correcto. Si todo esto se lleva a cabo satisfactoriamente, se procederá a realizar la inserción del nuevo socio.

Base de Datos 2

```
--=====
-- Carga de Socios
--=====

CREATE PROCEDURE sp_Agregar_Socio
@Nombre VARCHAR(100),
@Apellido VARCHAR(100),
@Direccion VARCHAR(255),
@Telefono VARCHAR(100),
@email VARCHAR(255)
AS BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION

            IF @Nombre IS NULL OR @Nombre = ''
                RAISERROR('El nombre no puede ser nulo ni vacío', 16, 1);

            IF @Apellido IS NULL OR @Apellido = ''
                RAISERROR('El apellido no puede ser nulo ni vacío', 16, 1);

            IF @Direccion IS NULL OR @Direccion = ''
                RAISERROR('La dirección no puede ser nula ni vacío', 16, 1);

            IF @Telefono IS NULL OR @Telefono = ''
                RAISERROR('El teléfono no puede ser nulo ni vacío', 16, 1);

            IF @Email IS NULL OR @Email = ''
                RAISERROR('El email no puede ser nulo ni vacío', 16, 1);

            -----
            -- VALIDAR DUPLICADOS (EMAIL)
            -----

            IF EXISTS(SELECT 1 FROM Socios WHERE Email = @Email)
                RAISERROR ('El email ingresado ya existe', 16, 1);

            -----
            -- VALIDAR FORMATO DE EMAIL
            -----

            IF @Email NOT LIKE '%@%.%'
                RAISERROR('El formato de email no es válido', 16, 1);

            -----
            -- INSERCIÓN SI TODO ES CORRECTO
            -----

            INSERT INTO Socios(Nombre, Apellido, Direccion, Telefono, Email)
            VALUES(@Nombre, @Apellido, @Direccion, @Telefono, @Email);

        COMMIT TRANSACTION
    END TRY

    BEGIN CATCH

        -----
        -- MANEJO DE ERRORES
        -----

        ROLLBACK TRANSACTION;
        PRINT 'ERROR AL AGREGAR SOCIO: ' + ERROR_MESSAGE();

    END CATCH
END
GO
```

Procedimiento Almacenado N°2 (Reporte Parametrizado):

Se utiliza el procedimiento almacenado “sp_Reporte_MultasPorSocio” para elaborar un reporte de las multas registradas para un socio específico,

Base de Datos 2

mostrando los datos del socio (ID del socio, Nombre y Apellido), el ID del préstamo, el ID de la multa, el monto de cada multa y la fecha de la multa. Además, mostrará un resumen con la cantidad total de multas, y el monto acumulado.

```
CREATE PROCEDURE sp_Reporte_MultasPorSocio
@IDSocio INT
AS BEGIN
    SET NOCOUNT ON;

    --=====
    -- Detalle de cada multa
    --=====

    SELECT
        s.IDSocio,
        s.Nombre,
        s.Apellido,
        p.IDPrestamo,
        m.IDMulta,
        m.Monto,
        m.FechaGenerada
    FROM Multas m
    INNER JOIN Prestamos p
        ON m.IDPrestamo = p.IDPrestamo
    INNER JOIN Socios s
        ON p.IDSocio = s.IDSocio
    WHERE s.IDSocio = @IDSocio;

    --=====
    -- Resumen total agrupado por socio
    --=====

    SELECT
        s.IDSocio,
        s.Nombre,
        s.Apellido,
        COUNT(m.IDMulta) AS [Cantidad de Multas],
        SUM(m.Monto) AS [Monto total]
    FROM Multas m
    INNER JOIN Prestamos p
        ON m.IDPrestamo = p.IDPrestamo
    INNER JOIN Socios s
        ON p.IDSocio = s.IDSocio
    WHERE s.IDSocio = @IDSocio
    GROUP BY s.IDSocio, s.Nombre, s.Apellido;
END
```

Función de usuario:

Esta función recibe el ID de un préstamo, y calcula la cantidad de días de atraso en la devolución del libro

```
CREATE FUNCTION fn_Dias_Atraso(@IDPrestamo INT)
RETURNS INT
AS BEGIN
    DECLARE @Dias INT;
    DECLARE @FechaEsperada DATE;
    DECLARE @FechaReal DATE;

    SELECT
        @FechaEsperada = FechaDevolucionEsperada,
        @FechaReal = FechaDevolucionReal
    FROM Prestamos
    WHERE IDPrestamo = @IDPrestamo;

    IF @FechaReal IS NULL
        SET @Dias = 0;

    ELSE IF @FechaReal <= @FechaEsperada
        SET @Dias = 0;

    ELSE
        SET @Dias = DATEDIFF(DAY, @FechaEsperada, @FechaReal);

    RETURN @Dias;
END
GO
```

Triggers

Al insertar Préstamos:

Si se intenta prestar más ejemplares de los que hay disponibles, se dispara el Trigger anulando el préstamo y realizando el rollback. Si por lo contrario hay stock suficiente para realizar el préstamo se realizará y se descontará el stock de ejemplares disponibles

Base de Datos 2

```
CREATE TRIGGER tr_Prestamos_AfterInsert
ON Prestamos
AFTER INSERT
AS
BEGIN
    BEGIN TRY
        -- 1) Buscar si para algún libro en 'inserted' se solicita más de lo disponible
        IF EXISTS (
            SELECT 1
            FROM (
                SELECT I.IDLibro, COUNT(*) AS Pedidos
                FROM inserted I
                GROUP BY I.IDLibro
            ) AS req
            JOIN Libros L ON L.IDLibro = req.IDLibro
            WHERE req.Pedidos > L.EjemplaresDisponibles
        )
        BEGIN
            RAISERROR('No hay suficientes ejemplares disponibles.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
        -- 2) Decrementar EjemplaresDisponibles por cada libro según la cantidad insertada
        UPDATE l
        SET l.EjemplaresDisponibles = l.EjemplaresDisponibles - r.CantidadPrestada
        FROM Libros l
        JOIN (
            SELECT i.IDLibro, COUNT(*) AS CantidadPrestada
            FROM inserted i
            GROUP BY i.IDLibro
        ) AS r ON l.IDLibro = r.IDLibro;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        RETURN;
    END CATCH
END
```

Al actualizar Préstamos:

Cuando se registra una devolución, se detecta la FechaDevolucionReal cambia de Null a NOT NULL, incrementa EjemplaresDisponibles por cada devolución y si la devolución es tardía, es decir la fecha de devolución es posterior a la fecha de devolución estimada, se insertará una fila en multas añadiendo el valor de 1.00 por cada día de atraso

Base de Datos 2

```
CREATE TRIGGER tr_Prestamos_AfterUpdate_Devolucion
ON Prestamos
AFTER UPDATE
AS
BEGIN
    BEGIN TRY
        -- Filas que pasaron de sin devolver (deleted.FechaDevolucionReal IS NULL)
        -- a devuelto (inserted.FechaDevolucionReal IS NOT NULL)
        ;WITH Devoluciones AS (
            SELECT i.IDPrestamo, i.IDLibro, i.FechaDevolucionEsperada, i.FechaDevolucionReal
            FROM inserted i
            JOIN deleted d ON d.IDPrestamo = i.IDPrestamo
            WHERE d.FechaDevolucionReal IS NULL AND i.FechaDevolucionReal IS NOT NULL
        )
        -- 1) Incrementar EjemplaresDisponibles por libro según cantidad de devoluciones
        UPDATE l
        SET l.EjemplaresDisponibles = l.EjemplaresDisponibles + d.CntDevoluciones
        FROM Libros l
        JOIN (
            SELECT IDLibro, COUNT(*) AS CntDevoluciones
            FROM Devoluciones
            GROUP BY IDLibro
        ) d ON l.IDLibro = d.IDLibro;

        -- 2) Generar multas para las devoluciones tardías
        DECLARE @TasaPorDia MONEY = 1.00;
        INSERT INTO Multas (IDPrestamo, Monto, FechaGenerada, Estado)
        SELECT
            dv.IDPrestamo,
            CAST(DATEDIFF(DAY, dv.FechaDevolucionEsperada, dv.FechaDevolucionReal) * @TasaPorDia AS MONEY) AS Monto,
            dv.FechaDevolucionReal AS FechaGenerada,
            CAST(1 AS BIT) AS Estado
        FROM Devoluciones dv
        WHERE DATEDIFF(DAY, dv.FechaDevolucionEsperada, dv.FechaDevolucionReal) > 0;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        RETURN;
    END CATCH
END
```

Impedir eliminación de Libros:

Los libros que tengan préstamos activos no podrán ser eliminados

que tengan préstamos activos, en el caso de que no tengan ningún préstamo activo sí podrán eliminarse

```
CREATE TRIGGER tr_Libros_PreventDelete_IfActiveLoans
ON Libros
AFTER DELETE
AS
BEGIN
    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM deleted d
            JOIN Prestamos p ON p.IDLibro = d.IDLibro
            WHERE p.FechaDevolucionReal IS NULL
        )
        BEGIN
            RAISERROR('No se puede eliminar el/los libro(s): existen préstamos activos asociados.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        RETURN;
    END CATCH
END
```

Base de Datos 2

Impedir eliminación de Socios:

Todos los socios que tengan préstamos activos no podrán ser eliminados, de lo contrario si podrán realizarlo

```
CREATE TRIGGER tr_Socios_PreventDelete_IfActiveLoans
ON Socios
AFTER DELETE
AS
BEGIN
    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM deleted d
            JOIN Prestamos p ON p.IDSocio = d.IDSocio
            WHERE p.FechaDevolucionReal IS NULL
        )
        BEGIN
            RAISERROR('No se puede eliminar el/los socio(s): existen préstamos activos asociados.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
    END CATCH
END
```

LINKS

[Link del repositorio en Github](#) -

<https://github.com/guillermoacampos/DBBiblioteca.git>

[link del video en Google Drive](#)